ChatGPT

面向 Docker 部署 AI 应用的人机协同智能预测运维平台：落地技术方案

执行摘要

本项目建议定位为一套**面向 Docker 化 AI 应用的单体式、模块化人机协同智能预测运维平台**：以 Prometheus 为指标与告警底座，以 `node_exporter + cAdvisor + Blackbox Exporter` 完成服务器、Docker 和服务可用性感知；以自研 LLM Probe 补足 DeepSeek API 的真实推理可用性监控；以三级预测体系完成“已发生故障 + 即将发生故障 + 异常行为”的统一识别；以 DeepSeek Harness 作为独立 **AI Engine**，负责事故上下文理解、知识检索、诊断、Skill 推荐和事故复盘；**所有有状态变更的 Action Skill 均不得由 LLM 直接执行，而必须进入平台自己的持久化人工审批流程，审批通过后才允许通过 SSH 执行。**

这一技术边界与现有生态是匹配的：Node Exporter 官方用于暴露 Linux 硬件和内核指标，cAdvisor 原生支持 Docker 并采集容器资源及网络数据；Blackbox Exporter 支持 HTTP/HTTPS、DNS、TCP、ICMP、gRPC 黑盒探测；Alertmanager 原生支持告警分组、去重、路由、邮件与 Webhook，因此没有必要重新实现底层监控告警系统。 [1]

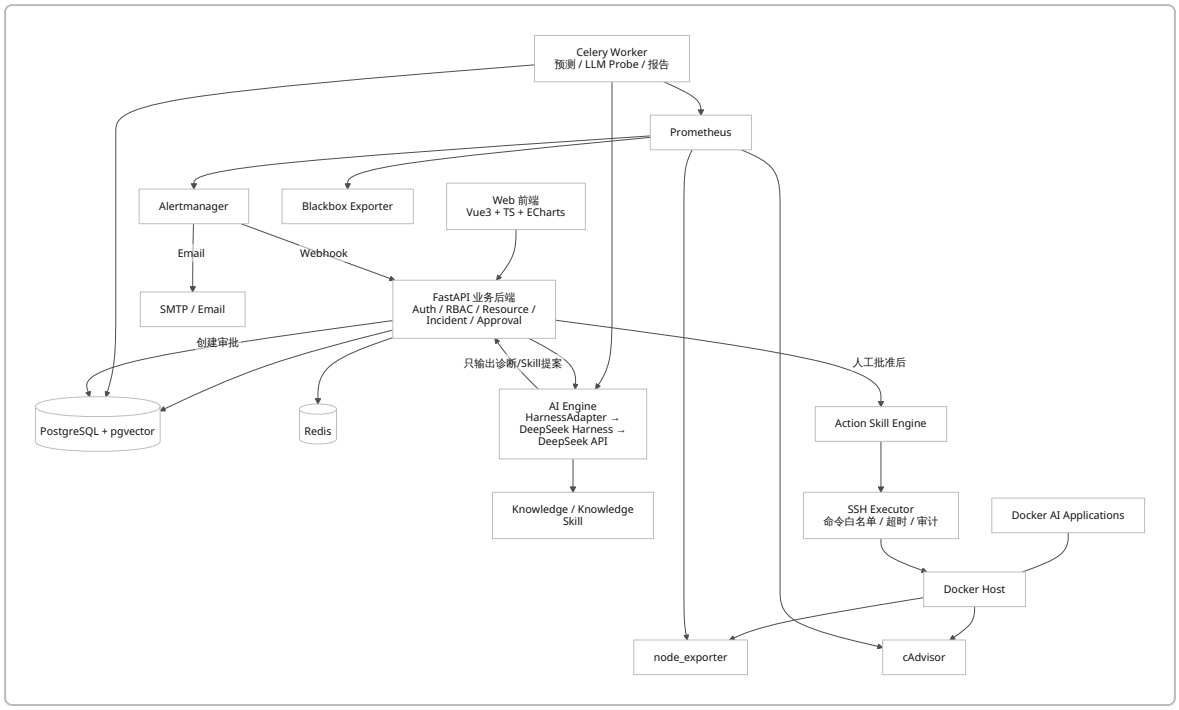
DeepSeek Harness 截至 **2026 年 10 月 1 日**已提供 Python SDK 和插件机制，因此可从 FastAPI 调用；但官方仍明确标注为 **Developer Preview**，并警告可能发生破坏兼容的变化。因此本方案不会让 Harness 成为平台核心业务框架，而通过 `HarnessAdapter` 隔离。更重要的是，官方 `sdk-minimal` 示例默认包含可修改运行环境的持久 Shell，并使用 `danger-full-access`；这进一步说明生产系统中不能直接把 Harness Shell 暴露给模型，而应只提供平台定义的受控工具。 [2]

严格边界：仅 Docker；SSH 可用；无 Kubernetes、无 CI/CD、无多 Agent、无无人值守自动修复；DeepSeek API 为 LLM；Agent/LLM 只辅助分析，人类拥有最终处置权。具体 DeepSeek 模型版本、SMTP 服务商、SSH 密钥托管方案、服务器规模、指标保存周期目前均为**未指定**。

总体架构与功能设计

总体架构

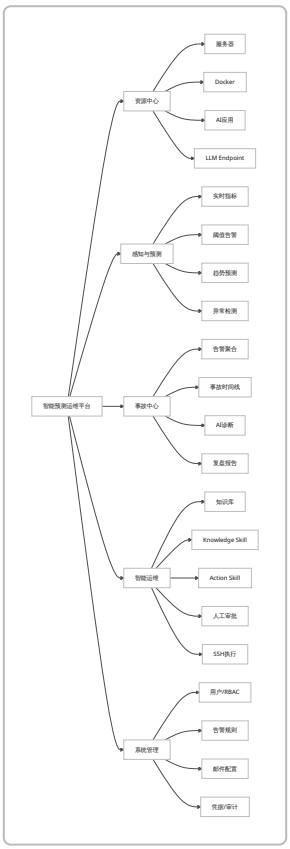
建议采用“**一个业务系统 + 一个 Prometheus 监控底座 + 一个 Harness AI Engine**”，首版不拆微服务。FastAPI 内部按领域模块化，Celery Worker 执行预测、LLM Probe、知识处理和异步报告任务。



Node Exporter 可提供 `node_cpu_seconds_total`、`node_filesystem_avail_bytes`、网络接收等 Linux 指标；cAdvisor 面向 Docker 持续采集、聚合并导出容器资源及网络统计。因此**日常监控不走 SSH**，SSH 只用于初始化、只读诊断以及审批后的 Action Skill，这是本方案的重要架构约束。 3

Blackbox Exporter 负责普通 HTTP/TCP 可达性，但 DeepSeek API 还应增加一个平台自己的 **LLM Probe**：周期性发起最小真实推理请求，记录成功率、总延迟、TTFT（流式时）、HTTP 状态码、Token 用量。
DeepSeek 官方当前区分 401 认证失败、402 余额不足、429 限流、500 内部故障和 503 过载，因此这些状态应映射为不同告警原因，而不是统称“LLM 不可用”。 4

功能模块



预测体系建议固定为三级，但三级的角色不同：

级别	职责	首版算法/方法	典型对象
Level A 阈值检测	判断“现在是否异常”	Prometheus Rules、持续时间窗口	CPU > 90%、容器 Down、LLM 503
Level B 趋势预测	判断“什么时候将异常”	`predict_linear`、线性回归、EWMA	磁盘耗尽、内存持续增长、带宽趋势
Level C 异常检测	判断“行为是否偏离历史”	Robust Z-Score；后续 Isolation Forest	网络突增、API 延迟突变、容器资源异常

Prometheus 官方 `predict_linear(v,t)` 本身利用简单线性回归预测 Gauge 的未来值，并明确要求用于 Gauge；同时 `/api/v1/query_range` 可以获取历史时间序列，因此首版没有必要建设另一套时序数据库。 5

告警链设计为：

`Prometheus Rule → Alertmanager → Email + Webhook → AlertEvent → Incident`

Alertmanager 自带 grouping、deduplication、routing、silence 和 inhibition；Receiver 同时支持 `email_configs` 与 `webhook_configs`，Webhook Payload 还带有 fingerprint，非常适合后台幂等落库。 6

数据库与 ER 设计

PostgreSQL 保存“资产、事件、决策和审计”，**不保存 Prometheus 原始时序数据**；pgvector 只用于知识库向量。字段类型以下为建议，精度、索引和数据保存周期均属实现参数，其中保存周期目前**未指定**。

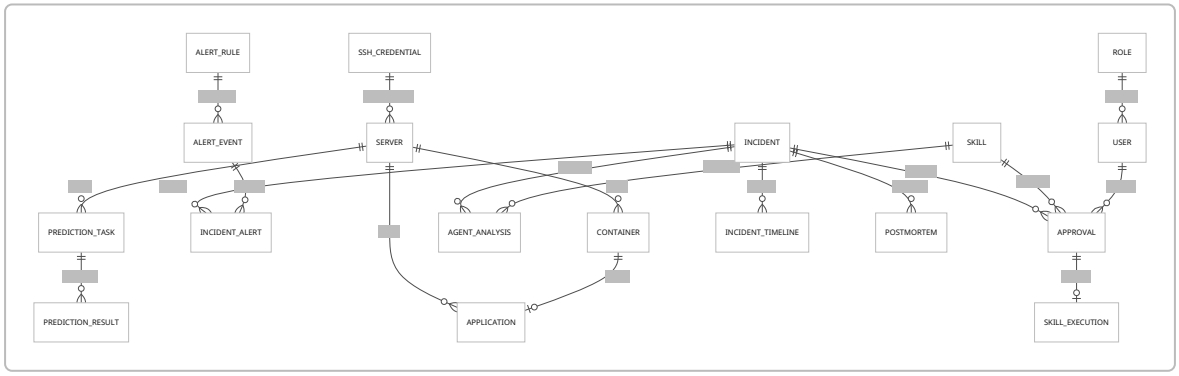
核心数据模型

表	核心字段
`users`	`id, username, email, password_hash, role_id, status, last_login_at, created_at`
`roles`	`id, code(viewer/operator/admin), name, permissions_json`
`servers`	`id, name, host, ssh_port, ssh_user, credential_id, environment, status, last_seen_at`
`ssh_credentials`	`id, name, auth_type, encrypted_secret, fingerprint, created_by`
`containers`	`id, server_id, docker_id, name, image, status, ports_json, labels_json, discovered_at`
`applications`	`id, name, server_id, container_id, health_url, owner_user_id, status`
`llm_endpoints`	`id, name, provider, base_url, model, credential_ref, probe_enabled, probe_interval, status`
`alert_rules`	`id, code, resource_type, severity, expression, for_duration, enabled, config_json`
`alert_events`	`id, fingerprint, alert_rule_id, resource_type, resource_id, severity, status, starts_at, ends_at, labels_json, annotations_json`
`prediction_tasks`	`id, resource_type, resource_id, metric, algorithm, lookback, horizon, schedule, enabled`
`prediction_results`	`id, task_id, predicted_value, risk_level, threshold, predicted_at, target_time, metadata_json`
`incidents`	`id, title, severity, status, resource_type, resource_id, started_at, recovered_at, assignee_id, root_cause`
`incident_alerts`	`incident_id, alert_event_id`
`incident_timeline`	`id, incident_id, event_type, actor_type, actor_id, content_json, created_at`
`knowledge_documents`	`id, title, source_type, content, embedding, tags_json, version, enabled`
`skills`	`id, name, type(KNOWLEDGE/ACTION), risk_level, description, definition_yaml, version, enabled`
`agent_analyses`	`id, incident_id, harness_session_id, summary, diagnosis_json, proposed_skill_id, confidence, created_at`

表	核心字段
`approvals`	`id, incident_id, skill_id, requested_by, parameters_json, status, approver_id, decision_comment, expires_at, decided_at`
`skill_executions`	`id, approval_id, server_id, skill_id, parameters_json, precheck_json, stdout_redacted, stderr_redacted, result, started_at, ended_at`
`postmortems`	`id, incident_id, content_md, ai_generated, reviewed_by, status, version, created_at`
`audit_logs`	`id, user_id, action, resource_type, resource_id, request_id, ip, detail_json, created_at`

`credentials` 表绝不能保存明文 API Key/SSH 私钥；具体采用 Vault、云 KMS 或本地 Master Key 的方案目前**未指定**。首版至少应做到“密文入库 + 解密主密钥与数据库分离”。

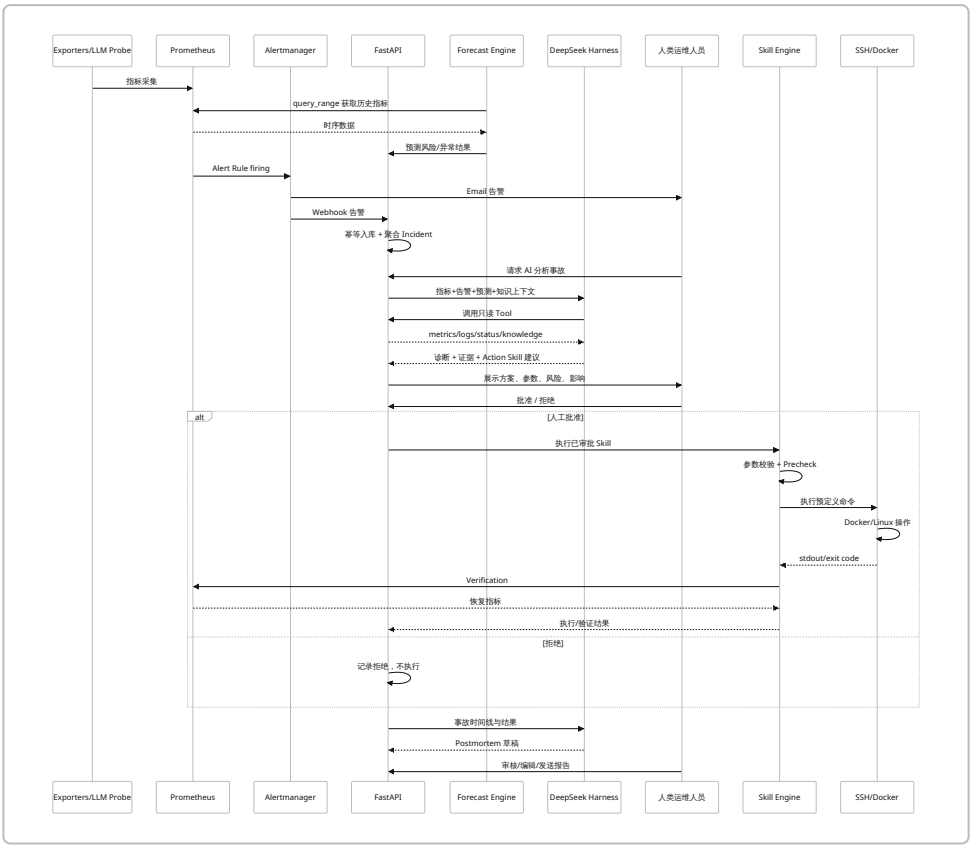
ER 关系



这里必须保持三个概念分离：**Alert 是监控事件，Incident 是业务事故，Approval 是执行授权**。例如内存、HTTP 和 LLM 调用异常可以被聚合进同一 Incident，但绝不能因为一个 Alert 到达就直接执行 SSH。

人机协同闭环与 DeepSeek Harness 设计

核心时序



Harness 的正确边界

Harness 官方 Python SDK 可以从 Python 创建 `DeepSeekHarness` 并调用 `run()`，同时支持 profile 和 plugin，因此与 FastAPI 集成在技术上成立。 [7]

但生产模式应明确禁止：

```
LLM → 任意 Bash → SSH → Server
```

而只允许：

```
LLM
  ├── get_server_metrics()       只读
  ├── get_container_status()     只读
  ├── get_recent_logs()          只读/脱敏
  ├── get_alert_context()        只读
  ├── search_knowledge()         只读
  ├── list_skills()              只读
  └── propose_action_skill()     仅生成提案
                          ↓
```

```
                平台持久化 Approval
                      ↓
                人工批准
                      ↓
                Skill Engine
                      ↓
                    SSH
```

这是因为官方 Python SDK 的 `sdk-minimal` 示例当前直接带持久化 Bash/PowerShell，且文档明确指出该 profile 使用 `danger-full-access`，可修改运行时可见路径；因此本项目应创建专属受限 profile，而不是照搬示例 profile。 [8]

此外，**平台自己的 `approvals` 表必须是审批事实源**。Harness 官方审批机制目前虽然是 fail-closed、one-shot 且会记录审计，但官方同时说明审批请求只能发生在 open turn 内，没有内建 answerer，而且 request 不携带 tool arguments；长期 Web 审批流程属于其当前限制。因此 Harness Approval 最多作为二次防线，不应该替代业务系统中可持续数分钟乃至数小时的审批单。 [9]

Skill 分层建议：

类型	内容	LLM 权限	示例
Knowledge Skill	SOP、判断步骤、故障模式、经验规则	可直接读取	`docker_oom_diagnosis`
Action Skill	参数 Schema、Precheck、固定 Action、Verify、Rollback、风险级别	只能提议	`restart_container`

Action Skill 不保存自由文本 Shell，而使用模板化动作，例如 `docker restart {{container_name}}`，其中 `container_name` 必须从平台资产中解析并校验，避免模型把任意文本转换成远程命令。

工程目录与 API 设计

前后端目录

```
ops-platform/
├── frontend/
│   └── src/
│       ├── api/                # REST client
│       ├── router/
│       ├── stores/
│       ├── layouts/
│       ├── views/
│       │   ├── dashboard/
│       │   ├── servers/
│       │   ├── containers/
│       │   ├── llm/
│       │   ├── monitoring/
│       │   ├── predictions/
│       │   ├── alerts/
```

```
│       │   ├── incidents/
│       │   ├── skills/
│       │   └── system/
│       └── components/charts/
│
├── backend/
│   └── app/
│       ├── main.py
│       ├── api/v1/
│       ├── core/               # config/security/RBAC
│       ├── models/             # SQLAlchemy
│       ├── schemas/            # Pydantic
│       ├── repositories/
│       ├── services/
│       │   ├── prometheus/
│       │   ├── alerting/
│       │   ├── prediction/
│       │   ├── incident/
│       │   ├── llm_probe/
│       │   ├── knowledge/
│       │   ├── harness/
│       │   │   └── adapter.py
│       │   ├── skill/
│       │   ├── approval/
│       │   ├── ssh/
│       │   └── postmortem/
│       ├── workers/
│       └── tests/
│
├── harness/
│   ├── profiles/ops-assistant/
│   ├── plugins/
│   └── knowledge-skills/
├── action-skills/
├── monitoring/
│   ├── prometheus/
│   ├── alertmanager/
│   └── blackbox/
└── docker-compose.yml
```

API 清单

认证机制此前**未指定**；首版建议 JWT Access/Refresh Token，角色沿用 `Viewer / Operator / Admin`。Alertmanager Webhook 不使用用户 JWT，而配置独立 Bearer Secret；Alertmanager HTTP receiver 官方支持 Authorization 配置。 10

方法与路径	请求 → 响应摘要	认证 / 最低权限
`POST /api/v1/auth/login`	`{username,password}` → `{access_token,refresh_token}`	Public
`GET /api/v1/me`	— → 当前用户	JWT / Viewer
`GET /api/v1/dashboard`	— → 资源、告警、事故、预测摘要	Viewer
`POST /api/v1/servers`	`{name,host,ssh_port,credential_id}` → Server	Admin
`GET /api/v1/servers`	filters → Server[]	Viewer
`GET /api/v1/servers/{id}`	— → ServerDetail	Viewer
`POST /api/v1/servers/{id}/test-ssh`	— → `{success,latency}`	Admin
`POST /api/v1/servers/{id}/bootstrap`	exporter 配置 → Task	Admin
`GET /api/v1/servers/{id}/metrics`	`metric,start,end,step` → Series[]	Viewer
`GET /api/v1/containers`	server/status filters → Container[]	Viewer
`GET /api/v1/containers/{id}`	— → 容器状态+指标	Viewer
`GET /api/v1/containers/{id}/logs`	`{since,tail}` → 脱敏日志	Operator
`POST /api/v1/llm-endpoints`	EndpointConfig → Endpoint	Admin
`POST /api/v1/llm-endpoints/{id}/probe`	— → ProbeResult	Operator
`GET /api/v1/llm-endpoints/{id}/metrics`	range → 成功率/延迟/错误码	Viewer
`GET /api/v1/alerts`	filters → Alert[]	Viewer
`POST /api/v1/hooks/alertmanager`	Alertmanager JSON → `{accepted:true}`	Webhook Secret
`POST /api/v1/alerts/{id}/ack`	`{comment}` → Alert	Operator
`GET /api/v1/predictions`	resource/metric → Result[]	Viewer
`POST /api/v1/prediction-tasks`	AlgorithmConfig → Task	Admin
`POST /api/v1/prediction-tasks/{id}/run`	— → PredictionResult	Operator
`GET /api/v1/incidents`	filters → Incident[]	Viewer
`GET /api/v1/incidents/{id}`	— → Incident+Timeline	Viewer

方法与路径	请求 → 响应摘要	认证 / 最低权限
`POST /api/v1/incidents/{id}/analyze`	`{force:false}` → AgentAnalysis	Operator
`GET /api/v1/incidents/{id}/analysis`	— → diagnosis/evidence/proposal	Viewer
`GET /api/v1/knowledge`	search/tag → Documents	Viewer
`POST /api/v1/knowledge`	文档/文本 → Document	Admin
`GET /api/v1/skills`	`type,risk` → Skill[]	Viewer
`POST /api/v1/skills`	Skill YAML/JSON → Skill	Admin
`POST /api/v1/incidents/{id}/approvals`	`{skill_id,parameters}` → Approval	Operator
`POST /api/v1/approvals/{id}/approve`	`{comment}` → Approved	Operator/ Admin*
`POST /api/v1/approvals/{id}/reject`	`{comment}` → Rejected	Operator/ Admin*
`POST /api/v1/approvals/{id}/execute`	— → Execution	Operator/ Admin*
`GET /api/v1/executions/{id}`	— → Precheck/Output/Verify	Viewer
`POST /api/v1/incidents/{id}/postmortem`	— → AI 草稿	Operator
`PUT /api/v1/postmortems/{id}`	`{content_md}` → Postmortem	Operator
`GET /api/v1/postmortems/{id}/export`	`format=pdf` → 文件	Operator
`POST /api/v1/postmortems/{id}/email`	`{recipients}` → SendResult	Admin
`GET /api/v1/audit-logs`	filters → AuditLog[]	Admin

`*` Action Skill 可再按风险设定审批策略，例如 Low=Operator，Medium/High=Admin；具体映射目前**未指定**。

典型 AI 分析请求与结果：

```
POST /api/v1/incidents/inc_123/analyze

{
  "force": false
}
```

```
{
 "incident_id": "inc_123",
 "summary": "ai-api 容器存在持续内存增长，并伴随 HTTP 延迟上升",
 "evidence": [
  "45 分钟内存使用率 68% -> 94%",
  "宿主机其他容器内存稳定",
  "HTTP P95 1.2s -> 4.7s"
 ],
 "possible_causes": ["应用内存泄漏或缓存未释放"],
 "proposal": {
  "skill": "restart_container",
  "parameters": {"container_id": "ctr_456"},
  "risk": "MEDIUM"
 },
 "requires_human_approval": true
}
```


注意这里返回的是 **proposal，不是 execution**。

第一阶段开发计划与验收

建议按“纵向闭环”而非“先把全部前端写完”的方式实施。以下估时基于 **2 名后端/平台开发 + 1 名前端 + 1 名算法/AI 可兼职，已有 Linux/Docker 测试服务器**；人员配置未由需求指定，因此时间仅作为规划假设。

里程碑	主要工作	验收标准	参考工期
基础骨架	Vue/FastAPI/PostgreSQL/Redis、用户、JWT、RBAC、Docker Compose	Admin/Operator/Viewer 可登录并按权限访问	3–4 天
资源与感知	Server、SSH 测试、node_exporter、cAdvisor、Prometheus、Blackbox	添加服务器后可看到 CPU/内存/磁盘/网络/容器实时曲线	5–7 天
告警闭环	Prometheus Rules、Alertmanager、Webhook、Email、AlertEvent	人为停止容器后形成告警、落库并收到邮件；恢复后状态关闭	4–5 天
LLM 监控	DeepSeek Endpoint、真实 Probe、状态码/延迟/成功率指标	模拟错误 Key/限流/服务异常时可区分告警原因	3–4 天
三级预测	阈值 + `predict_linear` /EWMA + Robust Z-Score	可展示磁盘/内存未来 1h/6h/24h 风险和异常点	5–7 天
Incident	Alert 聚合、状态机、Timeline、详情页	多个关联 Alert 可形成一个 Incident，并保留完整时间线	4–5 天
Harness + KB	HarnessAdapter、专用受限 Profile、RAG、只读 Tools、Knowledge Skill	AI 能根据指标/告警/知识生成有证据的诊断，不具备 SSH 执行能力	6–8 天
审批与 Action Skill	Skill Schema、Approval、SSH Executor、Precheck/Verify/Audit	AI 提议重启容器→人审批→SSH 执行→健康验证；未审批绝不能执行	6–8 天

里程碑	主要工作	验收标准	参考工期
复盘	AI Postmortem、编辑、PDF、管理员邮件发送	一次完整 Incident 可生成、审核、导出并发送事故报告	3–4 天

因此，一个功能闭环完整的第一阶段 MVP 可按约 **7–9 周**规划；这里比单纯监控项目多出的核心工作量主要位于 Incident、Harness 受控工具层、持久化审批、Skill Engine 与执行审计，而不是预测算法本身。

最终验收不应定义为“用了 Agent”，而应定义为以下闭环全部成立：

> **感知：**服务器、Docker、应用和 DeepSeek API 都能被监控
> **→ 预测：**平台能识别当前阈值风险、未来趋势风险和异常行为
> **→ 决策：**DeepSeek Harness 可以结合指标、告警、预测和知识生成诊断与 Skill 提案
> **→ 人工决策：**没有明确审批记录，任何 Action Skill 都不能执行
> **→ 执行：**审批后仅执行平台定义且参数校验过的 SSH Skill，并自动验证结果
> **→ 复盘：**全过程进入 Incident Timeline，由 LLM 生成事故报告、由人审核后导出或邮件发送。

从技术可行性看，Prometheus 生态已经覆盖本项目最重的“采集—告警—历史指标查询”基础设施：Node Exporter 负责主机，cAdvisor 负责 Docker，Blackbox 负责端点，Alertmanager 负责告警聚合与邮件/Webhook，Prometheus API/PromQL 支撑预测输入。[11] DeepSeek Harness 则适合被封装为可替换 AI Engine，其 Python SDK 和插件体系满足嵌入后端的需要，但由于官方仍处于 Developer Preview 且默认示例权限较高，**“业务系统掌握审批、Skill 和 SSH；Harness 只掌握分析和提案”应当成为项目不可破坏的架构原则。**[12]

---

[1] [3] [11] Monitoring Linux host metrics with the Node Exporter | Prometheus
https://prometheus.io/docs/guides/node-exporter/?utm_source=chatgpt.com

[2] [12] GitHub - deepseek-ai/deepseek-harness: DeepSeek Harness: Everything is a Plugin. · GitHub
https://github.com/deepseek-ai/deepseek-harness

[4] blackbox_exporter/README.md at master · prometheus/blackbox_exporter · GitHub
https://github.com/prometheus/blackbox_exporter/blob/master/README.md?utm_source=chatgpt.com

[5] Query functions | Prometheus
https://prometheus.io/docs/prometheus/latest/querying/functions/?utm_source=chatgpt.com

[6] Alertmanager | Prometheus
https://prometheus.io/docs/alerting/latest/alertmanager/

[7] deepseek-harness/docs/user/guide/python-sdk.md at master · deepseek-ai/deepseek-harness · GitHub
https://github.com/deepseek-ai/deepseek-harness/blob/master/docs/user/guide/python-sdk.md?utm_source=chatgpt.com

[8] deepseek-harness/docs/user/guide/python-sdk.md at master · deepseek-ai/deepseek-harness · GitHub
https://github.com/deepseek-ai/deepseek-harness/blob/master/docs/user/guide/python-sdk.md

[9] deepseek-harness/packages/interaction/user-approval/README.md at master · deepseek-ai/deepseek-harness · GitHub
https://github.com/deepseek-ai/deepseek-harness/blob/master/packages/interaction/user-approval/README.md?utm_source=chatgpt.com

10 Configuration | Prometheus
https://prometheus.io/docs/alerting/latest/configuration/